Custom HashMap Implementation - Hash Table Internals

Theory, Edge Cases & Code Examples

- ✓ Hash Table Basics
- ✓ Collision Resolution: Separate Chaining
- ✓ Load Factor and Rehashing
- ✓ Hash Function Design
- ✓ Rehashing Invalidates References

→ Swipe to learn →

Custom HashMap Implementation - Hash Table Internals

Theory

📌 1. Hash Table Basics

A **hash map** (hash table) stores key-value pairs using a hash function to compute array indices.

Components:

- Array of buckets
- Hash function: `hash(key) → index`
- Collision resolution strategy

Operations:

```
map.insert(key, value);  // O(1) average
value = map.find(key);   // O(1) average
map.erase(key);          // O(1) average
```

Custom HashMap Implementation - Hash Table Internals

Theory

📌 2. Collision Resolution: Separate Chaining

Separate chaining: Each bucket is a linked list.

```
Buckets:
[0] → (k1, v1) → (k5, v5)
[1] → (k2, v2)
[2] → empty
[3] → (k3, v3) → (k7, v7) → (k11, v11)
```

Insertion:

```
index = hash(key) % bucket_count
buckets[index].push_front({key, value})
```

Lookup:

```
index = hash(key) % bucket_count
for (entry in buckets[index]):
    if entry.key == key:
        return entry.value
```

Custom HashMap Implementation - Hash Table Internals

Theory

📌 3. Load Factor and Rehashing

Load factor: α = n / m (elements / buckets)

Optimal: α ≈ 0.75
- Too low: wasted space
- Too high: long chains, slow lookups

Rehashing: When α > threshold:
1. Allocate 2× larger array
2. Reinsert all elements (rehash)
3. Deallocate old array

Amortized O(1) insertion.

Custom HashMap Implementation - Hash Table Internals

Theory

📌 4. Hash Function Design (1/4)

Good hash function:

- Uniform distribution
- Fast to compute
- Deterministic

std::hash example:

```
std::hash<int>{}(42) → 42
std::hash<std::string>{}("hello") → large_number
```

Custom hash:

```
struct Person {
    std::string name;
    int age;
};

template<>
struct std::hash<Person> {
    size_t operator()(const Person& p) const {
        return std::hash<std::string>{}(p.name) ^
               (std::hash<int>{}(p.age) << 1);
    }
};
```

```
#include <vector>
#include <list>
#include <functional>
#include <stdexcept>
#include <utility>

template<typename Key, typename Value, typename Hash = std::hash<Key>>
class HashMap {
private:
    using Pair = std::pair<const Key, Value>;
    using Bucket = std::list<Pair>;

// ...continues on next slide
```

Custom HashMap Implementation - Hash Table Internals

Theory

📌 4. Hash Function Design (2/4)

```
// ...continued from previous slide
    std::vector<Bucket> buckets_;
    size_t size_;
    float max_load_factor_;
    Hash hasher_;

    size_t bucket_index(const Key& key) const {
        return hasher_(key) % buckets_.size();
    }

    void rehash(size_t new_bucket_count) {
        std::vector<Bucket> new_buckets(new_bucket_count);

        for (auto& bucket : buckets_) {
            for (auto& [key, value] : bucket) {
                size_t new_index = hasher_(key) % new_bucket_count;
                new_buckets[new_index].emplace_back(key, std::move(value));
            }
        }

        buckets_ = std::move(new_buckets);
    }

    void check_load_factor() {
        float load_factor = static_cast<float>(size_) / buckets_.size();
        if (load_factor > max_load_factor_) {
            rehash(buckets_.size() * 2);
        }
    }

public:
    HashMap(size_t initial_bucket_count = 16, float max_load = 0.75f)
        : buckets_(initial_bucket_count), size_(0), max_load_factor_(max_load) {}

    void insert(const Key& key, const Value& value) {
        size_t index = bucket_index(key);

        for (auto& [k, v] : buckets_[index]) {
            if (k == key) {
                v = value;  // Update existing
                return;
// ...continues on next slide
```

Custom HashMap Implementation - Hash Table Internals

Theory

📌 4. Hash Function Design (3/4)

```
// ...continued from previous slide
            }
        }

        buckets_[index].emplace_back(key, value);
        ++size_;
        check_load_factor();
    }

    bool find(const Key& key, Value& out_value) const {
        size_t index = bucket_index(key);

        for (const auto& [k, v] : buckets_[index]) {
            if (k == key) {
                out_value = v;
                return true;
            }
        }

        return false;
    }

    Value& operator[](const Key& key) {
        size_t index = bucket_index(key);

        for (auto& [k, v] : buckets_[index]) {
            if (k == key) {
                return v;
            }
        }

        buckets_[index].emplace_back(key, Value{});
        ++size_;
        check_load_factor();

        return buckets_[index].back().second;
    }

    bool erase(const Key& key) {
        size_t index = bucket_index(key);

// ...continues on next slide
```

Custom HashMap Implementation - Hash Table Internals

Theory

📌 4. Hash Function Design (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        auto& bucket = buckets_[index];
        for (auto it = bucket.begin(); it != bucket.end(); ++it) {
            if (it->first == key) {
                bucket.erase(it);
                --size_;
                return true;
            }
        }

        return false;
    }

    size_t size() const { return size_; }
    bool empty() const { return size_ == 0; }

    float load_factor() const {
        return static_cast<float>(size_) / buckets_.size();
    }
};
```

Custom HashMap Implementation - Hash Table Internals

Edge Cases

📌 1. Rehashing Invalidates References

```
HashMap<int, int> map;
map[1] = 10;
int& ref = map[1];  // Reference to value

map.insert(many_keys...);  // Triggers rehash
ref = 20;  // ← INVALID (dangling reference)
```

Custom HashMap Implementation - Hash Table Internals

Edge Cases

📌 2. Custom Hash Collisions

```
struct BadHash {
    size_t operator()(int x) const { return 0; }  // All collide!
};

HashMap<int, int, BadHash> map;  // O(n) operations
```

Custom HashMap Implementation - Hash Table Internals

Edge Cases

📌 3. Key Not Found

```
int value;
if (map.find(key, value)) {
    // Use value
} else {
    // Handle missing key
}
```

Custom HashMap Implementation - Hash Table Internals

Code Examples

📌 1. Word Frequency Counter (1/2)

This example demonstrates using a custom hash map to count word frequencies, showcasing the convenience of operator[] for automatic initialization and increment operations.

What this code does:

- Creates a hash map that associates each string (word) with an integer (frequency count)
- Processes a vector of words containing duplicates
- Uses operator[] which automatically creates entries with Value{} (0 for int) for new keys
- Increments the count for each word occurrence using the convenient map[word]++ syntax
- Queries the final count for a specific word using the find() method

Key concepts demonstrated:

- operator[] provides map semantics similar to std::unordered_map (auto-insertion on missing keys)
- Default-constructed Value{} initializes integers to 0, making counting logic simple
- The hash map handles collision resolution transparently via separate chaining
- find() returns bool to safely distinguish between "key exists" and "key missing"
- No manual resize needed - the map automatically rehashes when load factor exceeds 0.75

Real-world applications:

- Text analysis and natural language processing (word frequency analysis)
- Log file analysis to count error types or user actions
- Network packet analysis to track source IP addresses or protocol types
- Game statistics tracking (counting enemy kills, item pickups, etc.)

Why this matters:

- Average O(1) lookup and insertion makes this efficient even for large datasets
- Automatic rehashing ensures consistent performance as the dataset grows
- operator[] simplifies counting patterns (no need to check if key exists first)
- Separate chaining gracefully handles hash collisions without complex probing logic

Performance implications:

- Each word lookup is O(1) average case, making the entire loop O(n) where n is number of words
- Hash function quality matters - std::hash provides good distribution
- Short chains (load factor 0.75) keep lookup fast even with collisions
- String hashing has overhead - for extreme performance, consider integer IDs instead

---

Custom HashMap Implementation - Hash Table Internals

Code Examples

📌 1. Word Frequency Counter (2/2)

```
HashMap<std::string, int> word_count;

std::vector<std::string> words = {"hello", "world", "hello", "test"};

for (const auto& word : words) {
    word_count[word]++;
}

int count;
if (word_count.find("hello", count)) {
    std::cout << "hello appears " << count << " times\n";  // 2
}
```

What You Learned

Key Takeaways

- ✓ Hash Table Basics
- ✓ Collision Resolution: Separate Chaining
- ✓ Load Factor and Rehashing
- ✓ Hash Function Design

→ Swipe to learn →